

- 01 — Core Concepts (No Code)
 - 1. What is an "agent"?
 - 2. What is MCP (Model Context Protocol)?
 - The USB analogy
 - Why two processes?
 - 3. What is LangGraph?
 - The factory assembly line
 - What lives on the conveyor belt?
 - The specialist team
 - 4. What is a tool?
 - 5. The request lifecycle (cartoon version)
 - 6. Key vocabulary cheat sheet
 - 7. Why is this hard?

01 — Core Concepts (No Code)

Goal: After reading this page you should be able to explain MCP, LangGraph, and the agent loop to a non-programmer using analogies.

1. What is an "agent"?

In normal software, you write code that tells the computer exactly what to do:

```
Step 1: Read sensor X
Step 2: If value > 100, print "ALERT"
```

An **agent** flips this around. You give it a **goal** ("Analyze Mill 8 for anomalies") and the agent decides the steps itself. It can:

- Ask questions
- Run database queries
- Write Python scripts
- Decide when it is done

Think of it like giving a task to a smart intern instead of a calculator.

2. What is MCP (Model Context Protocol)?

The USB analogy

Your computer has USB ports. You can plug in a mouse, a keyboard, or a hard drive. The computer does not need to know how the mouse works inside; it just uses the standard USB protocol.

MCP is a USB port for AI tools.

In our project:

- The **mouse/keyboard** = tools like `query_mill_data`, `execute_python`, `write_markdown_report`
- The **computer** = the AI agent (LangGraph)
- The **USB port** = MCP

Because we use MCP, we could unplug our tool set and plug it into Claude Desktop, another LangChain app, or a CLI inspector — and everything would still work.

Why two processes?

The system runs as **two separate Python programs**:

1. **MCP Server** (`server.py`, port 8003) — owns the tools, the database connection, and the DataFrames.
2. **FastAPI process** (`api.py`, port 8000) — owns the web API and LangGraph.

Why not one program? Because the tool server is **stateful** (it keeps DataFrames in memory), while the API is **stateless** (each request is independent). Keeping them separate means you can restart the API without losing loaded data, and you can connect other clients to the same tool server.

3. What is LangGraph?

The factory assembly line

Imagine a car factory:

- **Frame station** welds the chassis
- **Paint station** paints the body
- **Engine station** installs the motor
- **Quality station** checks everything

A **conveyor belt** moves the half-built car between stations.

LangGraph works exactly like this:

- **Station** = a node (e.g., `data_loader`, `analyst`, `reporter`)
- **Conveyor belt** = the `state` object that carries messages and data between nodes
- **Factory controller** = the edges and routing logic that decide which station comes next

What lives on the conveyor belt?

The `state` is just a Python dictionary that grows as the factory progresses:

```
state = {
    "messages": [      # every conversation between agents and tools
        HumanMessage("Analyze mill 8"),
        AIMessage("I'll load the data first"),
        ToolMessage("Here are 4,000 rows"),
        ...
    ],
    "current_stage": "analyst",
    "stages_to_run": ["analyst", "forecaster", "reporter"],
}
```

Every time a node finishes, it appends new messages to `state["messages"]`. The next node reads those messages to understand what happened before it.

The specialist team

Instead of one AI doing everything, we have a **team of specialists**:

Specialist	Job description
data_loader	"Go to the database, fetch the raw numbers, and tell me what you see."
planner	"Based on the user's question, which specialists should we call?"
analyst	"Calculate statistics, draw distribution plots, build SPC charts."
forecaster	"Predict the next 8 hours using Prophet."
anomaly_detective	"Find weird spikes and explain why they happened."
shift_reporter	"Compare KPIs across shifts and write a handover summary."
critic	"Look at all charts and verify the numbers are correct."
reporter	"Write the final Markdown report in Bulgarian."

A **manager** reviews each specialist's work. If the manager says "REWORK", the specialist tries again (once). If the manager says "ACCEPT", the conveyor belt moves to the next station.

4. What is a tool?

A **tool** is just a Python function that the AI is allowed to call.

Example: `execute_python`

- **Input:** a string of Python code
- **What it does:** runs the code in a safe namespace with pandas, numpy, matplotlib, etc.
- **Output:** stdout + any saved chart files

The AI does not "know" how to run Python. It only knows:

1. There is a tool called `execute_python`
2. It needs one argument: `code`
3. The result will be text

When the AI decides to use the tool, LangGraph pauses the conveyor belt, runs the function, packs the result into a `ToolMessage`, and puts it back on the belt so the AI can read it.

5. The request lifecycle (cartoon version)

```
User: "Hey, check Mill 8!"
```

```
UI:      "Sure, sending to API..."
```

```
API:      "Got it. Starting background task. Here is an ID: ab12cd34"
```

```
UI:      "Thanks, I'll check every 4 seconds."
```

```
LangGraph:
```

```
  data_loader: "Loading 4,000 rows from the DB..."
```

```
  planner:     "We need analyst + anomaly_detective."
```

```
  analyst:     "Here are the charts. [calls execute_python]"
```

```
  manager:     "ACCEPT."
```

```
  anomaly_detective: "Found 3 anomalies. [calls execute_python]"
```

```
  manager:     "ACCEPT."
```

```
  critic:      "Numbers look correct."
```

```
  reporter:    "Writing final report... [calls write_markdown_report]"
```

```
  manager:     "ACCEPT. DONE!"
```

```
API:      "Status = completed. Report is ready."
```

```
UI:      "Great! I'll fetch the Markdown and the PNG charts."
```

```
User sees: a chat bubble with a full report and a gallery of charts.
```

6. Key vocabulary cheat sheet

Term	Meaning
Agent	An AI that decides its own steps to reach a goal.
Tool	A function the AI can call (e.g., run Python, query DB).

Term	Meaning
Node	One step in the LangGraph pipeline (e.g., <code>analyst</code>).
Edge	The arrow between nodes; decides where to go next.
State	The shared dictionary that carries data between nodes.
Router	A function that looks at the state and picks the next node.
Message	A single entry in the conversation (Human, AI, Tool, System).
Prompt	The hidden instructions given to the AI at each node.
Context window	How much text the AI can read at once (~20 messages for us).
Progress callback	A function that tells the UI what the agents are doing right now.

7. Why is this hard?

Three challenges make this system tricky:

1. **The AI forgets.** It can only read ~20 recent messages. We solve this with `build_focused_context` — a function that throws away old raw output and keeps only summaries.
2. **The AI can loop forever.** A specialist might keep calling `execute_python` endlessly. We solve this with an iteration cap (max 5 tries per specialist).
3. **The AI can hallucinate.** It might invent numbers. We solve this with the `critic` stage, which double-checks charts and statistics.

Next step: Read `02_architecture.md` to see which process holds which data, then `03_mcp_deep_dive.md` to see the actual code behind the USB port.